Ops Analyst Agent
Personal project · Asil Burak Can

Live demo ops-analyst-agent.streamlit.app
Code github.com/asilburakcan/Ops-Analyst-Agent
Project page asilburakcan.github.io/Ops-Analyst-Agent

Ops Analyst Agent: case study

Asil Burak Can · personal project · AWS, Python, LightGBM, LLM agents

0.80	5.4%	12 / 12	~10 min
model AUC on 13 unseen days (rules: 0.59 / 0.58)	fail rate in the flagged 1%, about 9× the 0.62% average	SQL agent golden set, reference SQL match	server time per day; a few cents a month

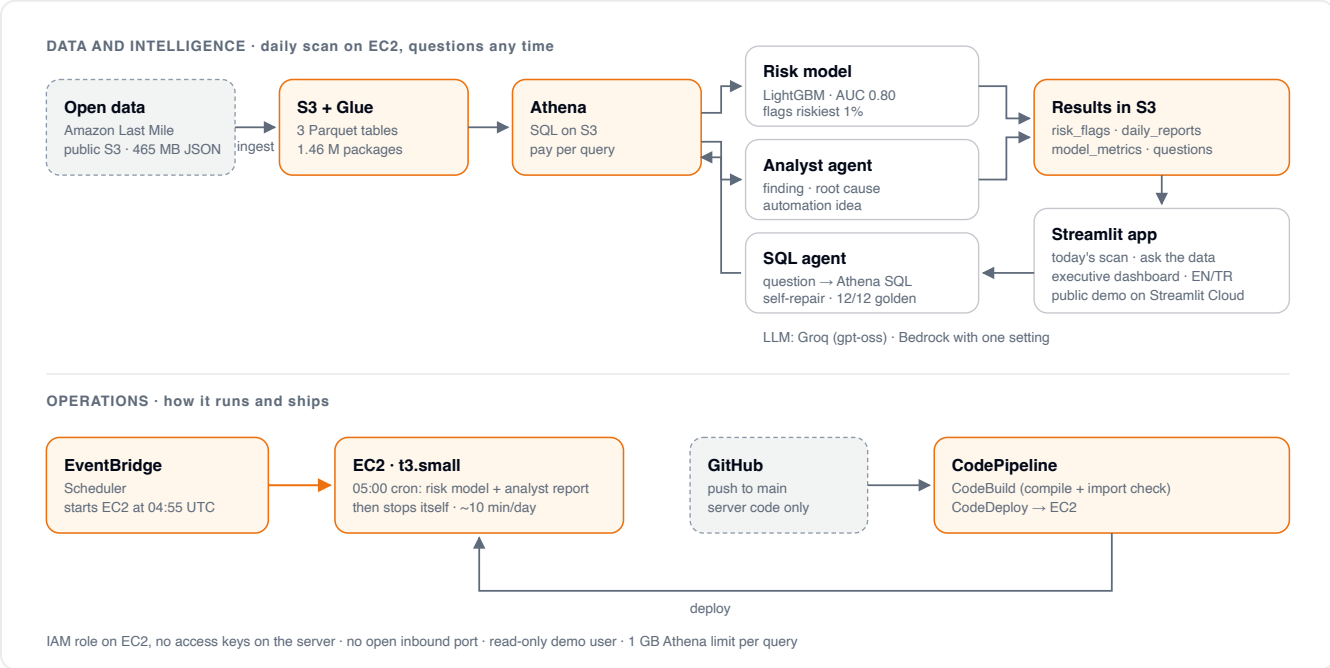
1. The problem

A failed delivery costs a second attempt, a longer route and an unhappy customer. In a typical station, dispatchers spot risky packages from experience, and operations questions ("which station fails most, and why?") wait for someone who can write SQL. I wanted to see how much of this can be automated on AWS from data that already exists, with every claim checked against ground truth rather than trusted.

2. The data

The 2021 Amazon Last Mile Routing Research Challenge dataset holds 6,112 real routes from 17 stations in 5 US metro areas, driven in July–August 2018. `ingest.py` reads the nested JSON from the public bucket and writes three Parquet tables to S3, registered in Glue: 6,112 routes, 904,527 stops and 1,457,175 packages. The run stops if a package has no stop, a stop has no visit order or a row is duplicated. Two real facts are kept and reported instead of removed: 6,515 dropoffs have no zone, and 1,422 packages appear at two stops of one route (attempted at one, delivered at the other). 0.76% of all packages fail.

3. What I built



Daily scan on EC2; questions any time through Athena; server-code changes ship through CodePipeline.

- **Risk model.** LightGBM on features known before the van leaves: station, departure hour, weekday, route size and load, packages at the stop, delivery window and the time left to it, planned service time and parcel

volume. Route score and visit order exist only after the route, so they are never used. The newest 20% of days are held out as "today".

- **Analyst agent.** Reads the scan and fixed comparison tables (flagged vs not flagged, fail rate by delivery window, by service-time quartile, top stations) and writes a finding, a root-cause hypothesis and an automation idea.
- **SQL agent.** Turns a question into one Athena `SELECT`, runs it, and repairs it up to two times if it fails or returns nothing.
- **App.** Streamlit in the AWS console style: today's scan, ask the data, executive dashboard, English and Turkish.

4. Results

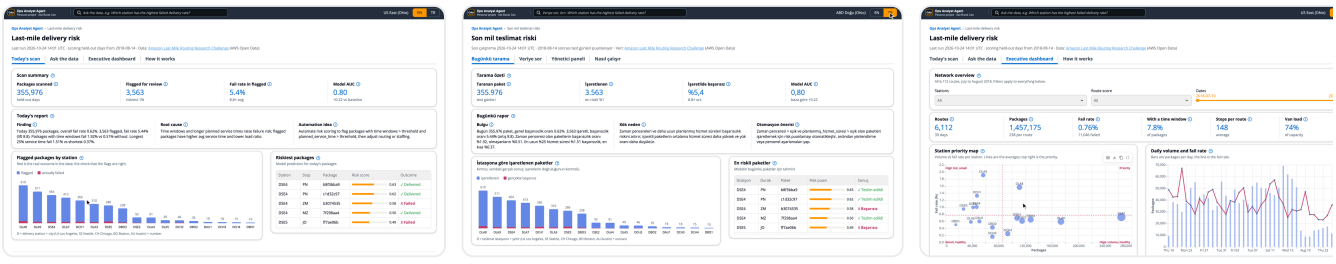
Check	Result	Why it matters
Model AUC on 13 held-out days (355,976 packages)	**0.80**	The two simple rules, the station's past fail rate and "has a delivery window", reach 0.59 and 0.58.
Fail rate in the flagged 1% (3,563 packages)	**5.4%** vs 0.62%	A reviewer who checks the flagged list finds about 9 times more failures than in a random sample.
Accuracy	not used	With 0.62% failures, "everything gets delivered" scores 99.38% and helps no one.
SQL agent golden set	**12 / 12**, 0 repairs	12 questions with reference SQL: joins, grouping, NULLs, percentages and business wording.

The model's strongest signals are planned service time at the stop, packages per stop, the station, the time left to the delivery window and route size. The analyst's report points to the same cause with numbers: packages with a delivery window fail 1.92% vs 0.51% on the held-out days, and the longest quarter of service times fails 1.51% vs 0.37% for the shortest.

5. Keeping the agents honest

Most of the work was not prompting but closing the gaps where a language model guesses. Every error I found became a golden-set question, so it is checked on every change.

- **One glossary for people and agents.** `DATA_DICTIONARY.md` defines package, stop, station, failed delivery and the rest. Both agents read its data terms; only the analyst reads the model terms. The SQL agent once read "station" as the depot stop type; the glossary and a golden question fixed it.
- **Causes only from comparisons.** The analyst once blamed weather, which is not in the data. Now a root cause must come from a comparison table; if it cannot, the agent says so and names the comparison to run.
- **Wrong answers count as failures.** An empty result triggers a repair, so a wrong filter is fixed instead of reported as "no data".
- **Read-only by design.** A single `SELECT` only; write requests are refused before any query or LLM call.



Today's scan · Executive dashboard · Turkish interface

6. Operations and cost

- EventBridge Scheduler starts EC2 at 04:55 UTC; cron runs the scan at 05:00; the instance stops itself. About 10 minutes of server time a day.
- CodePipeline runs only when server code changes: CodeBuild compiles and imports every module, CodeDeploy installs on EC2.
- IAM role on the server, no access keys, no open port. The public demo uses a read-only user, a 1 GB Athena limit per query and 50 questions a day.
- Cost: a few cents a month. Data in S3 is 124 MB; Athena charges $5 per TB scanned.

7. Limits and next steps

- **Historical data.** Each run scores the same 13 held-out days. Connecting a live feed only changes the ingest step.
- **Validation.** One time split today; rolling time-series validation and calibrated probabilities would make the risk score usable as a probability.
- **Action.** Send the flagged list to dispatch (SNS or email) and track how many flags were acted on, so the model is judged on prevented failures, not only on AUC.
- **Thresholds.** A fixed 1% is simple; a per-station capacity for reviews would match how stations actually work.

Personal portfolio project, not affiliated with Amazon. Data: Amazon Last Mile Routing Research Challenge, AWS Open Data, CC BY-NC 4.0. Code: MIT.